



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

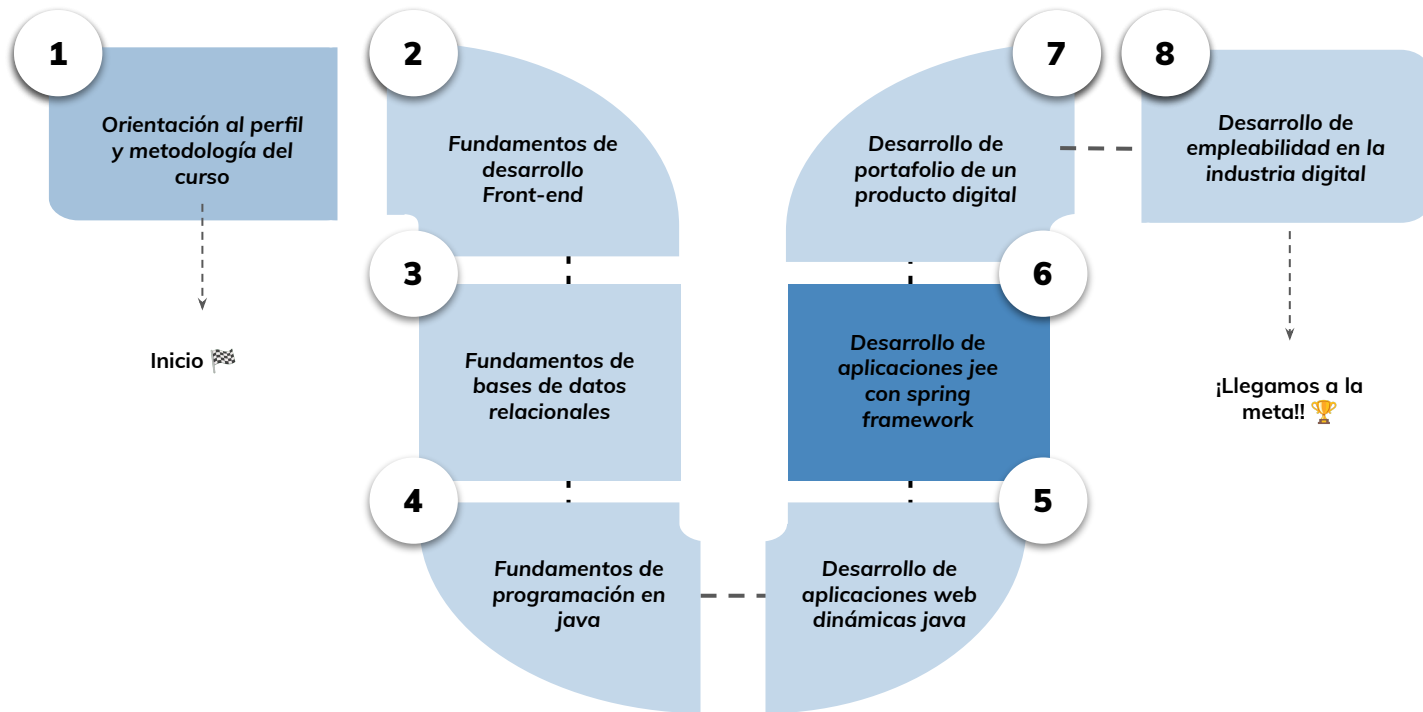


› El framework spring MVC - Parte I

Aprendizaje Esperado 2: Utilizar framework spring boot y spring mvc para la implementación de una aplicación web básica de acuerdo a las buenas prácticas de la industria.

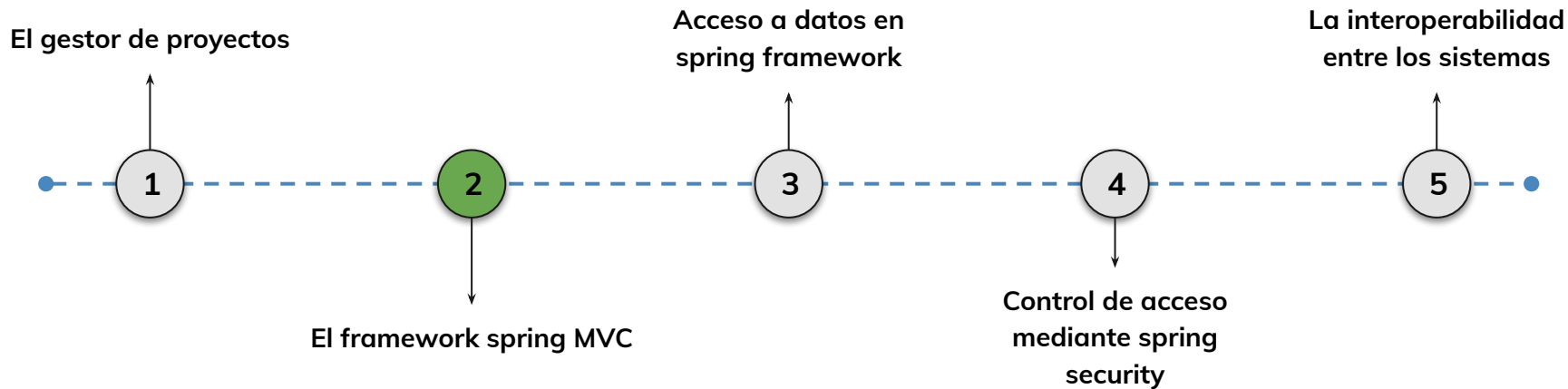
Hoja de ruta

¿Cuáles skills conforman el programa? **Desarrollo de Aplicaciones Full Stack Java Trainee**



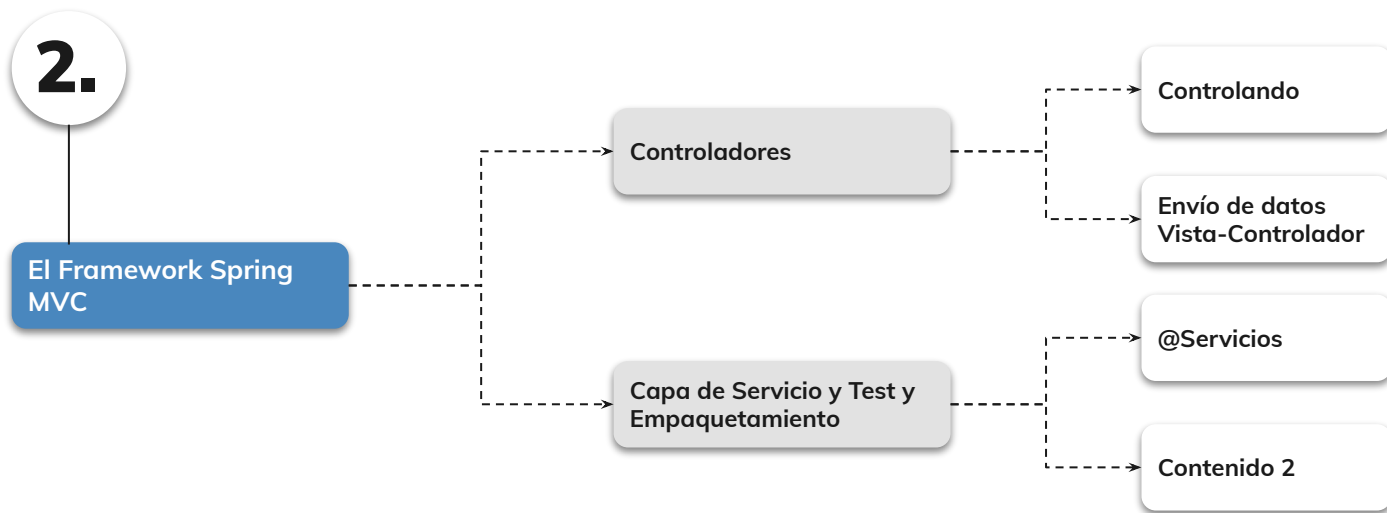
Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?



Objetivos de aprendizaje

¿Qué aprenderás?

- Conocer la programación en capas con Vistas y Controladores
- Comprender la configuración de peticiones
- Aprender a recibir datos en el controlador y a enviar datos a la vista
- Realizar el despliegue de una vista JSP con datos entregados en el Controlador
- Aprender a aplicar la capa de servicios en proyectos Spring
- Comprender la inyección de servicios en los controladores
- Comprender el proceso de test y empaquetamiento en Spring
- Aprender a crear unidades de prueba

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Conocer el framework Spring Boot y sus características
- Aprender sobre anotaciones
- Aprender sobre anotaciones e inyección de dependencias
- Conocer Spring Boot Initializr y sus características
- Aprender la estructura de un proyecto Spring
- Conocer la configuración de las tecnologías de vista
- Aprender a configurar un datasource

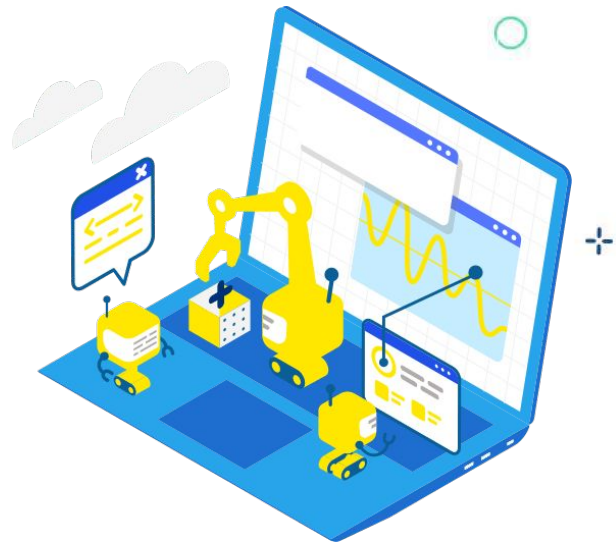
› Controladores



Controladores

Un controlador en Java, específicamente en el contexto de desarrollo web, es una componente esencial que forma parte del patrón de diseño

Modelo-Vista-Controlador (MVC) . Un controlador se encarga de manejar las solicitudes del cliente, procesarlas y decidir cómo responder.





Controladores



Los **Controllers** son las clases Java que usaremos como controladores en nuestra aplicación web. Estas clases son muy parecidas a los Servlets de Java EE pero más sencillas de usar. Recuerda que usamos estas clases gracias a que estamos usando el framework Spring MVC (habiendo incluido la dependencia spring-web en nuestro proyecto).



No es necesario que los controladores en Spring MVC extiendan de ninguna otra clase ni que implementen ninguna interfaz. Únicamente deben incluir la anotación **@Controller** al principio de la clase.





Controladores

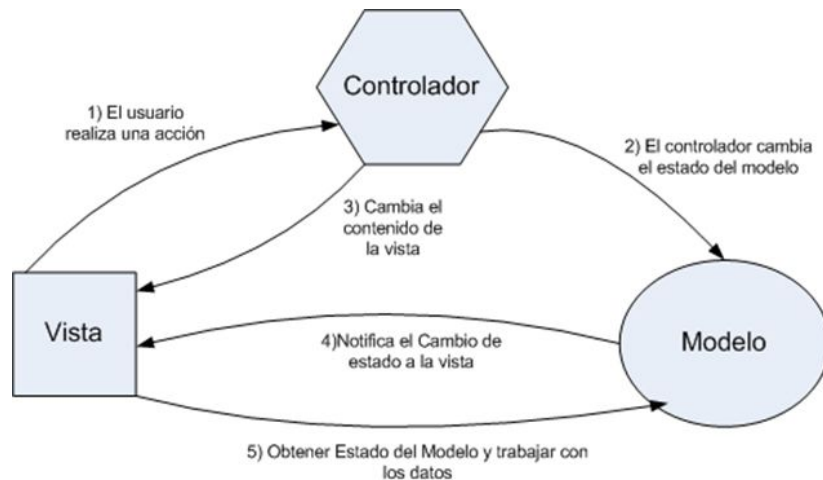


Funciones y Responsabilidades :

- **Gestión de solicitudes** : son responsables de recibir las solicitudes HTTP entrantes, como solicitudes de página, envíos de formularios o llamadas a servicios web.
- **Procesamiento de solicitudes** : lo que puede incluir validación de datos, ejecución de la lógica de negocio y acceso a servicios o recursos necesarios.
- **Selección de modelo y vista** : Basado en la solicitud y el resultado del procesamiento, el controlador decide qué modelo utilizar y qué vista mostrar como respuesta.
- **Preparación de datos para la vista:** Los controladores pueden cargar datos desde el modelo y prepararlos para que sean representados en la vista. Esto puede incluir la recuperación de datos de la base de datos o la interacción con otros servicios.
- **Devolver respuestas al cliente:** retornan respuestas al cliente en forma de vistas, objetos JSON, archivos u otros tipos de datos, según la solicitud y los resultados del procesamiento.



Controladores





Controladores

Vamos a analizar por ejemplo el siguiente código



```
13 @Controller
14 @RequestMapping("/{index.html"})
15 public class ProfessorController {
16     @Autowired
17     private ProfesorDAO profesorDAO;
18
19
20     public ModelAndView read(HttpServletRequest request, HttpServletResponse response) {
21         Map<String, Object> model = new HashMap<>();
22         String viewName;
23
24         try {
25             Profesor profesor = profesorDAO.get(1001);
26             model.put("texto", profesor.toString());
27             viewName = "profesor";
28         } catch (Exception ex) {
29             model.put("msgError", "No es posible obtener los datos");
30             viewName = "error";
31         }
32
33         return new ModelAndView(viewName, model);
34     }
35 }
36 }
```





Controladores

Como ya hemos dicho lo principal que debe tener un controller de Spring MVC es la anotación **@Controller** al principio de la clase (Línea 13). Con ésto Spring MVC sabrá que esta clase es un bean de tipo controlador.



```
13 @Controller
14 @RequestMapping("/{index.html}")
15 public class ProfesorController {
16     @Autowired
17     private ProfesorDAO profesorDAO;
18
19
20     public ModelAndView read(HttpServletRequest request, HttpServletResponse response) {
21         Map<String, Object> model = new HashMap<>();
22         String viewName;
23
24         try {
25             Profesor profesor = profesorDAO.get(1001);
26             model.put("texto", profesor.toString());
27             viewName = "profesor";
28         } catch (Exception ex) {
29             model.put("msgError", "No es posible obtener los datos");
30             viewName = "error";
31         }
32
33         return new ModelAndView(viewName, model);
34     }
35 }
36 }
```





Controladores

Manejo de peticiones

El **manejo de peticiones web se realiza a través de los métodos del controlador** . Cada método gestionará las peticiones de una o más URLs y puede haber tantos métodos como se desee. Ésto es una gran ventaja respecto a un Servlet ya que hace mucho más sencillo procesar una serie de URL relacionadas en un único controlador. Recuerda que en un Servlet había un único método para todas las peticiones.





Controladores

Cada método que controla peticiones Web tendrá la anotación **@RequestMapping** (Línea 14). Esta anotación tendrá como argumento un Array de Strings con todas las URLs que va a manejar. Por ello nótese que hay una llave “{ }” antes y después del String de la URL que maneja, ya que se permite más de una.



```
13 @Controller
14 @RequestMapping("/{index.html"})
15 public class ProfesorController {
16     @Autowired
17     private ProfesorDAO profesorDAO;
18
19
20     public ModelAndView read(HttpServletRequest request, HttpServletResponse response) {
21         Map<String, Object> model = new HashMap<>();
22         String viewName;
23
24         try {
25             Profesor profesor = profesorDAO.get(1001);
26             model.put("texto", profesor.toString());
27             viewName = "profesor";
28         } catch (Exception ex) {
29             model.put("msgError", "No es posible obtener los datos");
30             viewName = "error";
31         }
32
33         return new ModelAndView(viewName, model);
34     }
35
36 }
```



Controladores

Los argumentos de entrada del método son los objetos **HttpServletRequest** y **HttpServletResponse** al igual que en un Servlet.

Spring MVC tiene muchas funcionalidades relativas a las URL que controla: **parámetros de entrada y de salida**.

El método retorna una clase de Spring MVC llamada **ModelAndView**. Esta clase contendrá un String con la página **jsp** a mostrar (es decir la **Vista**) y un **Map** con los datos que necesita la vista para mostrarse (es decir el **Modelo**).





Controladores

Podemos ver en la línea 33:
`return new ModelAndView(viewName, model);` como se crea un nuevo objeto **ModelAndView** que tiene como argumentos un String con el nombre de la página **JSP** a mostrar y el **Map** con los datos para la vista.



```
13 @Controller
14 @RequestMapping({"/index.html"})
15 public class ProfessorController {
16     @Autowired
17     private ProfesorDAO profesorDAO;
18
19
20     public ModelAndView read(HttpServletRequest request, HttpServletResponse response) {
21         Map<String, Object> model = new HashMap<>();
22         String viewName;
23
24         try {
25             Profesor profesor = profesorDAO.get(1001);
26             model.put("texto", profesor.toString());
27             viewName = "profesor";
28         } catch (Exception ex) {
29             model.put("msgError", "No es posible obtener los datos");
30             viewName = "error";
31         }
32
33         return new ModelAndView(viewName, model);
34     }
35
36 }
```



Controladores

Beneficios de un controlador en Java:



- **Separación de preocupaciones** : La arquitectura MVC, con un controlador en el centro, permite una clara separación de las preocupaciones de presentación, lógica de negocio y acceso a datos. Esto hace que el código sea más mantenible y escalable. +
- **Reutilización de código** : Los controladores permiten reutilizar la lógica de negocio en diferentes vistas y acciones. Esto evita la duplicación de código y promueve la coherencia en la aplicación.
- **Flexibilidad y extensibilidad** : Al utilizar un controlador, puedes cambiar fácilmente la vista que se muestra sin modificar la lógica subyacente. También es más sencillo +
agregar nuevas funcionalidades a la aplicación sin afectar otras partes del sistema.



Controladores



- **Pruebas unitarias** : Los controladores son componentes que se pueden probar de manera aislada, lo que facilita la creación de pruebas unitarias y la verificación de que la lógica funcione correctamente.
- **Escalabilidad:** Al separar las preocupaciones y utilizar un controlador, es más sencillo escalar una aplicación, ya que puedes agregar nuevos controladores y vistas según sea necesario.
- **Mantenimiento más sencillo** : La estructura organizada y la separación de preocupaciones hacen que el mantenimiento de la aplicación sea más manejable, ya que es más fácil identificar y solucionar problemas.



➤ Vista JSP



Vista JSP



La página JSP

Las páginas JSP se deben colocar en la carpeta `src/main/resources/templates`.

La vista tiene acceso a los elementos del Map del modelo que se creó en la clase ModelAndView mediante el siguiente código:



```
Object texto=request.getAttribute("texto");
```

Siendo el String “texto” la clave del Map con la que se guardó el valor.





Vista JSP



Vemos en la línea 3 cómo se accede a los datos del modelo.

```
1 <%@page contentType="text/html" pageEncoding="ISO-8859-1"%>
2 <%
3   Object texto = request.getAttribute("texto");
4   %>
5   <!DOCTYPE html>
6   <html>
7   <head>
8     <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
9     <title>Profesor</title>
10  </head>
11  <body>
12    <h1><%=texto%></h1>
13  </body>
14  </html>
```

➤ Controladores Multiacción



Controladores Multiacción



En el contexto de Spring MVC, un controlador **multiacción** se refiere a una técnica en la cual **un único controlador puede manejar múltiples acciones** (métodos) basados en diferentes URL o parámetros de solicitud. Esto permite agrupar varias acciones relacionadas en un solo controlador, lo que puede mejorar la organización y la legibilidad del código.



A menudo se utilizan en aplicaciones web para simplificar la gestión de múltiples acciones relacionadas, como operaciones CRUD (Crear, Leer, Actualizar, Eliminar) en recursos específicos.





Controladores Multiacción



Características de los Controladores Multiacción :

1. **Acciones basadas en parámetros** : En lugar de asociar una ruta específica a cada método en un controlador, los controladores multiacción analizan los parámetros de la solicitud para determinar qué acción debe llevarse a cabo. 
2. **Centralización de la lógica** : Las acciones relacionadas se agrupan en una única clase de controlador, lo que facilita la organización y el mantenimiento del código.
3. **Reducción de duplicación de código** : Al permitir que un solo método maneje varias acciones, se evita la duplicación de código similar en diferentes métodos de controlador.





Controladores Multiacción



Beneficios de los Controladores Multiacción :

- **Simplificación de rutas** : Se pueden utilizar rutas más generales en las solicitudes, lo que puede mejorar la legibilidad de las URL y simplificar la configuración de rutas. +
- **Centralización de la lógica de control** : La lógica relacionada se encuentra en un solo lugar, lo que facilita la comprensión y el mantenimiento del código.
- **Menos código repetitivo** : Al manejar varias acciones en un solo método, se reduce la necesidad de duplicar código similar en diferentes métodos de controlador.





Controladores Multiacción

Supongamos que estamos desarrollando una aplicación web para administrar una lista de tareas. En lugar de crear un método de controlador separado para "crear tarea", "actualizar tarea" y "eliminar tarea", podríamos usar un controlador multiacción.



```
@Controller
public class TareasController {

    @PostMapping("/tareas")
    public String gestionarTarea(@RequestParam String accion) {
        if ("crear".equals(accion)) {
            // Lógica para crear una tarea
        } else if ("actualizar".equals(accion)) {
            // Lógica para actualizar una tarea
        } else if ("eliminar".equals(accion)) {
            // Lógica para eliminar una tarea
        }
        // Redirigir a la página de tareas
        return "redirect:/tareas";
    }
}
```





Controladores Multiacción

En este ejemplo, el controlador **TareasController** maneja las acciones relacionadas con las tareas. El parámetro en la solicitud determina si se debe crear, actualizar o eliminar una tarea. La lógica de cada acción se maneja dentro del mismo método, lo que simplifica el código y centraliza la gestión de tareas.



```
@Controller
public class TareasController {

    @PostMapping("/tareass")

    public String gestionarTarea(@RequestParam String accion) {
        if ("crear".equals(accion)) {
            // Lógica para crear una tarea
        } else if ("actualizar".equals(accion)) {
            // Lógica para actualizar una tarea
        } else if ("eliminar".equals(accion)) {
            // Lógica para eliminar una tarea
        }
        // Redirigir a la página de tareas
        return "redirect:/tareass";
    }
}
```





Controladores Multiacción

Consideraciones y Mejores Prácticas :



- Si bien los pueden simplificar la gestión de múltiples acciones, es importante mantenerlos organizados y asegurarse de que cada acción tenga su propia lógica específica.
- Debes tener cuidado de no sobrecargar un controlador con demasiadas acciones, ya que esto podría complicar la legibilidad y el mantenimiento del código.
- Si las acciones se vuelven complejas o numerosas, considera la posibilidad de dividir el controlador en múltiples clases para mantener la organización y la claridad del código.
- La elección de utilizar controladores multiacción o controladores convencionales depende de la arquitectura de tu aplicación y de tus preferencias personales. Ambos enfoques



tienen sus propias ventajas y desventajas.

LIVE CODING

WalletController:

Vamos a comenzar el diseño de una Billetera Virtual para un cliente bancario que desea brindar una nueva experiencia a sus usuarios, incluyendo en el catálogo de beneficios una Wallet donde puedan gestionar algunas de las principales transacciones bancarias.

Los usuarios deben poder ver su saldo disponible, realizar depósitos y retiros de fondos.

Tiempo: 20 minutos

LIVE CODING

1. En el proyecto AlkeWallet, crear una clase Cuenta
2. Agregar los atributos String nombreUsuario, Integer numeroCuenta, Double saldoActual, String verDatos
3. Crear una clase CuentaController
4. Crear los métodos verSaldo, depositar, retirar y verDatos
5. Crear una vista "cuenta.jsp"
6. Mostrar, a modo de ejemplo, como se puede enviar el saldo del usuario a la vista.



Ejercicio

Controlando



Controlando

Consigna:

Vamos a hacer un pequeño ejercicio de prueba para aplicar las anotaciones e importaciones necesarias.

En la carpeta `src/main/java`, desde el paquete principal (`com.example.demo`) crea un nuevo paquete llamado “controladores” (`com.example.demo.controladores`).

En este nuevo paquete debes:

1. Crear una clase `@Controller` que responda a una petición `@RequestMapping("/index")`
2. Crear un método que retorne un String “nombre”

Tiempo : 10 minutos

➤ Recibiendo datos en el controlador



Recibiendo datos

Una de las tareas principales de un controlador Spring es recibir datos provenientes del cliente, generalmente a través de una solicitud **HTTP**. Esto puede incluir parámetros, rutas, encabezados u otros datos adjuntos a la solicitud. Spring proporciona varias formas de recibir estos datos en el controlador.





Recibiendo datos

1- **Parámetros de Solicitud** : Los controladores Spring pueden recibir parámetros de solicitud que se envían en la URL o en el cuerpo de una solicitud HTTP. Esto se logra mediante la anotación `@RequestParam`. Por ejemplo, supongamos que tienes una solicitud HTTP que envía un parámetro llamado "nombre":



```
@GetMapping("/saludo")
public String saludar(@RequestParam("nombre") String nombre) {
    // Procesar el parámetro "nombre"
    return "saludo";
}
```





Recibiendo datos

2- **Variables de Ruta** : Los controladores también pueden recibir datos a través de variables de ruta, las cuales se identifican con la anotación `@PathVariable`. Esto es útil para extraer valores de la URL de la solicitud. Por ejemplo, si tienes una URL como "/usuario/123", puedes recibir el valor "123" como una variable de ruta:



```
@GetMapping("/usuario/{id}")
public String verUsuario(@PathVariable("id") Long id) {
    // Procesar el ID del usuario
    return "perfil";
}
```





Recibiendo datos

3- **Objetos de Solicitud** : Spring permite recibir un objeto completo de solicitud, que incluye todos los datos de la solicitud, como parámetros, encabezados y más. Esto se hace utilizando la anotación `@ModelAttribute`. Por ejemplo:



```
@PostMapping("/nuevo-usuario")
public String crearUsuario(@ModelAttribute Usuario usuario) {
    // Procesar el objeto de solicitud (Usuario)
    return "confirmacion";
}
```





Recibiendo datos

4- **Objeto HttpServletRequest:** proporciona acceso a todos los detalles de una solicitud HTTP, incluyendo parámetros, encabezados, cuerpo de la solicitud y otros atributos relacionados con la solicitud.



```
@GetMapping("/ejemplo")
public String recibirDatos(HttpServletRequest request) {
    // Recibir datos de la solicitud utilizando HttpServletRequest
    String parametroNombre = request.getParameter("nombre");
    String parametroEdad = request.getParameter("edad");

    // Procesar los datos
    // ...

    return "vista";
}
```





Recibiendo datos

Sin embargo, es importante destacar que el uso de `HttpServletRequest` a menudo implica una codificación más detallada y manual en comparación con el uso de anotaciones como `@RequestParam` o `@PathVariable`, que simplifican la extracción de datos de la solicitud. Por lo tanto, se recomienda utilizar `HttpServletRequest` **cuando sea necesario**, pero considera usar anotaciones **cuando sea posible** para una implementación más concisa y mantenible.



➤ Entregando datos a la vista



Entregando datos a la vista

Una vez que el controlador ha procesado los datos de la solicitud, el siguiente paso es entregar esos datos a la vista para que se muestren al usuario final.

En Spring, este proceso se realiza mediante la devolución de un modelo de vista, que contiene los datos necesarios para la vista. Spring ofrece varias formas de hacerlo:





Entregando datos a la vista



1- **ModelAndView** :: Ya hemos visto ejemplos de cómo se utiliza en la explicación anterior. En este enfoque, el objeto **ModelAndView** se utiliza para combinar tanto la vista a la que se debe redirigir como el modelo de datos que se desea entregar a esa vista.

```
@GetMapping("/perfil")
public ModelAndView verPerfil() {
    ModelAndView modelAndView = new ModelAndView("perfil");
    modelAndView.addObject("nombre", "Juan");
    modelAndView.addObject("edad", 30);
    return modelAndView;
}
```



En este ejemplo, "perfil" es el nombre de la vista a la que se redirigirá, y se agregan atributos como "nombre" y "edad" al modelo de datos. Estos atributos estarán disponibles para su uso en la vista "perfil".



Entregando datos a la vista



2- **La interfaz Model** se utiliza para agregar atributos al modelo de datos en Spring. La principal ventaja de este enfoque es que no es necesario crear un objeto ModelAndView. Spring se encarga de seleccionar automáticamente la vista basándose en el nombre del controlador.



```
@GetMapping("/perfil")
public String verPerfil(Model model) {
    model.addAttribute("nombre", "Juan");
    model.addAttribute("edad", 30);
    return "perfil";
}
```

En este caso, simplemente devolvemos el nombre de la vista, "perfil". Spring sabe que debe buscar una vista llamada "perfil" para renderizar.



Entregando datos a la vista



3- **Map**: Aunque es menos común que Model o ModelAndView, es útil cuando se necesita una flexibilidad extrema en la construcción del modelo.

```
@GetMapping("/perfil")
public String verPerfil(Map<String, Object> model) {
    model.put("nombre", "Juan");
    model.put("edad", 30);
    return "perfil";
}
```



Este enfoque permite que cualquier objeto sea colocado en el modelo utilizando su nombre como clave en el mapa.



Entregando datos a la vista

4- . **JSON o XML (ResponseBody)** : Si necesitas entregar datos en formato **JSON** o **XML**, Spring facilita la conversión automática de objetos a estos formatos utilizando la anotación **@ResponseBody**. Esto es útil cuando se desarrollan servicios web RESTful y se necesita transmitir datos en lugar de vistas.



```
@GetMapping("/datos-json")
@ResponseBody
public Persona obtenerDatosEnJSON() {
    Persona persona = new Persona("Juan", 30);
    return persona;
}
```

En este ejemplo, la anotación **@ResponseBody** indica que el método del controlador debe devolver el objeto **Persona** como una respuesta en formato **JSON**. Spring se encarga de la serialización y envío de datos al cliente.



Entregando datos a la vista

5- **Uso de Thymeleaf y JSP** :. Estos motores de plantillas permiten que los datos entregados desde el controlador se integren de manera dinámica en el contenido HTML. Por ejemplo, en una plantilla Thymeleaf:

```
<p>Nombre: [[${nombre}]]</p>
<p>Edad: [[${edad}]]</p>
```

Aquí, los atributos "nombre" y "edad" se insertan en el HTML utilizando la sintaxis de Thymeleaf, lo que permite una presentación dinámica de los datos en la vista.



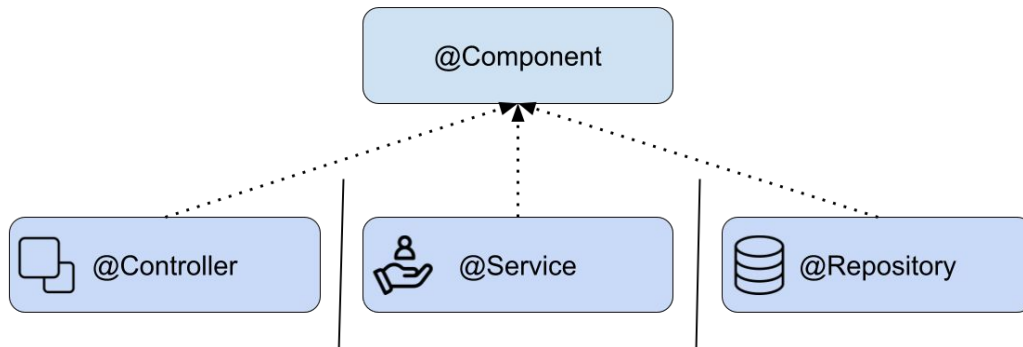
➤ Capa de Servicios



Capa de Servicios



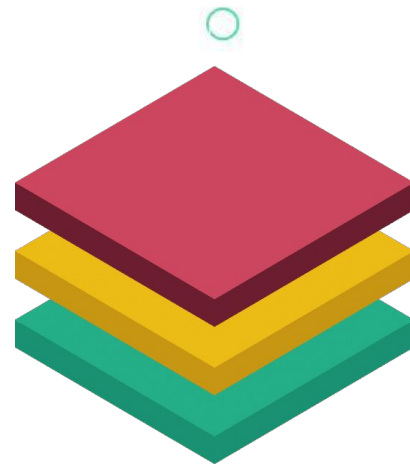
En el patrón de arquitectura **Modelo -Vista -Controlador** (MVC), la **capa de servicio** (también conocida como capa de negocio o capa control) es responsable de **implementar la lógica del negocio** y coordinar las operaciones entre la capa de presentación (vista) y la capa de acceso a datos (modelo).





Capa de Servicios

El propósito principal de la capa de servicio es **encapsular la lógica del negocio y proporcionar una interfaz que la capa de presentación pueda utilizar para acceder y manipular los datos** . Esta capa se encarga de procesar las solicitudes recibidas desde la capa de presentación, realizar las operaciones requeridas y devolver los resultados correspondientes.



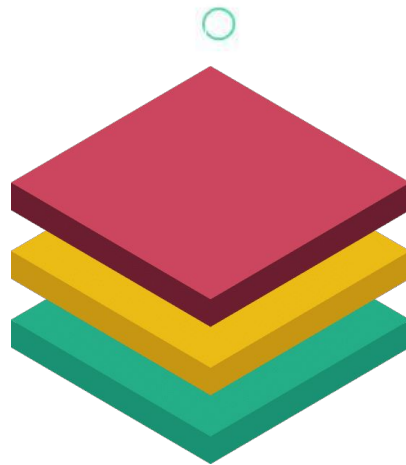


Capa de Servicios

Algunas de las **responsabilidades** típicas de la capa de servicio son:

Validación de datos : La capa de servicio valida los datos ingresados por el usuario antes de realizar cualquier operación en la capa de acceso a datos. Esto garantiza que los datos sean consistentes y cumplan con las reglas de negocio establecidas.

Lógica de negocio : Implementa la lógica del negocio de la aplicación. Esto puede incluir cálculos complejos, verificaciones de seguridad, reglas de flujo de trabajo, procesamiento de pagos, integraciones con servicios externos, entre otros.

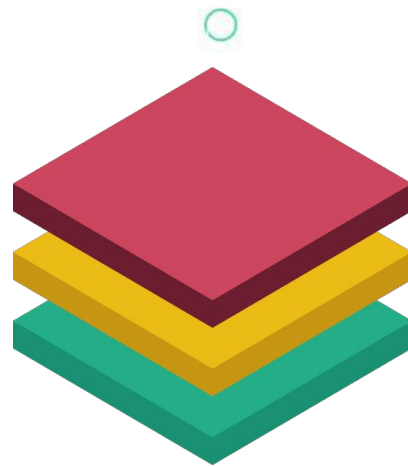




Capa de Servicios

Coordinación de la capa de acceso a datos : La capa de servicio interactúa con la capa de acceso a datos para realizar operaciones de lectura y escritura en la base de datos u otros sistemas de almacenamiento. Puede orquestar múltiples operaciones de acceso a datos y garantizar la integridad de los datos.

Transacciones y gestión de errores : La capa de servicio puede administrar transacciones, asegurando la integridad de los datos en operaciones que requieren cambios en varias entidades o tablas. También puede manejar excepciones y errores, proporcionando mecanismos para recuperarse de fallos y comunicar mensajes de error adecuados a la capa de presentación.



LIVE CODING

Wallet Service:

Las clases de servicio deberán tener su propio paquete en nuestro proyecto. Vamos a continuar el diseño de la AlkeWallet creando el paquete 'servicios' en la carpeta `src/main/java`.

Además, vamos a crear una clase llamada 'UsuarioServicio' que manipulará la lógica de los usuarios.

1. Crear el paquete `.service`
2. Agregar al paquete la clase `UsuarioServicio`

 **Tiempo:** 10 minutos



➤ Creando un servicio utilizando anotaciones



Creando un servicio utilizando anotaciones

`@Service` es una de las anotaciones más habituales de Spring Framework . Se usa para construir una clase de Servicio que habitualmente se conecta a varios repositorios y agrupa su funcionalidad.

Veamos el ejemplo de la clase que creamos recién, "`UsuarioServicio`", que implementa la lógica de negocio para manejar operaciones relacionadas con usuarios.





Creando un servicio utilizando anotaciones



Para convertir esta clase en un servicio utilizando anotaciones deberías hacer lo siguiente:

1- Anotar la clase : Agrega la anotación `@Service` en la clase para indicar que es un servicio.



```
3 import org.springframework.stereotype.Service;
4
5 @Service
6 public class UsuarioServicio {
7
8 }
9
```



Creando un servicio utilizando anotaciones

2- Inyectar dependencias: Si el servicio necesita utilizar otras clases o componentes, puedes inyectarlos utilizando la anotación `@Autowired`.

En este ejemplo, se inyecta un objeto `Usuario` en el servicio. La anotación `@Autowired` se encargará de proporcionar automáticamente una instancia de `Usuario` cuando se cree una instancia del servicio.

```
7
8 @Service
9 public class UsuarioServicio {
10
11     @Autowired
12     private Usuario usuario;
13 }
14
```

LIVE CODING

Wallet Service:

- Vamos a anotar la clase 'UsuarioServicio' con @Service.
- Además, debemos anotar con @Autowired un atributo del tipo Usuario. De ésta manera, podemos colocar los métodos de creación de usuarios en este servicio.
 1. Anotar como @Service la clase UsuarioServicio
 2. Inyectar una instancia de Usuario con @Autowired
 3. Generar la lógica de **creación** de usuarios en la clase UsuarioServicio.

Tiempo: 20 minutos

➤ Inyectando el servicio a un controlador para su utilización

Inyectando el servicio a un controlador para su utilización

Puedes utilizar el servicio en otras partes de tu aplicación inyectándolo de manera similar a como se hizo con las dependencias. Por ejemplo, puedes inyectar el servicio en un controlador utilizando la anotación **@Autowired**:

```
8 @Controller
9 public class UsuarioControlador {
10
11     @Autowired
12     private UsuarioServicio usuarioServicio;
13 }
```



Inyectando el servicio a un controlador para su utilización



Puedes utilizar el servicio en otras partes de tu aplicación inyectándolo de manera similar a como se hizo con las dependencias. Por ejemplo, puedes inyectar el servicio en un controlador utilizando la anotación **@Autowired**:



En este caso, el servicio `UsuarioServicio` se inyecta en el controlador `UsuarioControlador` para poder utilizar sus métodos y lógica de negocio.

```
8 @Controller
9 public class UsuarioControlador {
10
11     @Autowired
12     private UsuarioServicio usuarioServicio;
13 }
```

LIVE CODING

Wallet Service:

- Ahora vamos a inyectar un objeto del tipo `UsuarioServicio` en el controlador. De esta manera podremos utilizar las llamadas a los métodos CRUD del servicio desde el controlador.
- Además, debemos crear un método en el servicio que retorne un `String` con el nombre del usuario.

Tiempo: 20 minutos

LIVE CODING

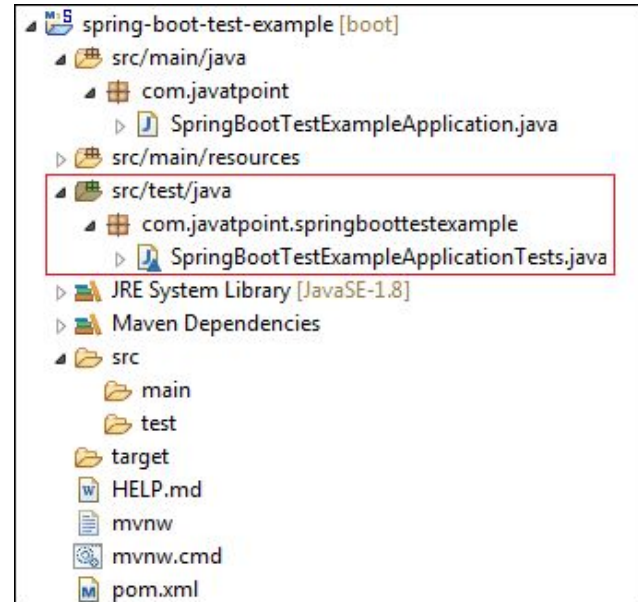
1. Inyectar una instancia de `UsuarioServicio` en el controlador de usuarios.
2. Generar un método denominado “respuesta” que retorne el nombre del usuario en la clase `UsuarioServicio`.
3. Llamar al método ‘respuesta’ desde el controlador utilizando la instancia de `usuarioServicio`.
4. Mostrar el nombre por pantalla.

➤ Test y empaquetamiento



Test y empaquetamiento

Las pruebas y el empaquetamiento son dos aspectos esenciales en el desarrollo de aplicaciones Spring. Las pruebas te permiten asegurarte de que tus componentes funcionen correctamente, mientras que el empaquetamiento es crucial para la distribución y el despliegue de tu aplicación.



Test

Pruebas en Spring:

Las pruebas en Spring son fundamentales para garantizar que tus componentes, clases y métodos funcionen de acuerdo a lo previsto. Existen diferentes tipos de pruebas que puedes realizar en un entorno Spring:



Test

1. Pruebas Unitarias:

Las pruebas unitarias son pruebas que se centran en unidades individuales de código, como clases o métodos. En Spring, puedes realizar pruebas unitarias utilizando frameworks de pruebas como **JUnit** o **TestNG**. A menudo, estas pruebas se realizan en aislamiento, lo que significa que no se interactúa con componentes externos, como bases de datos o servicios web.

Spring brinda anotaciones como **@RunWith(SpringRunner.class)** y **@SpringBootTest** para facilitar la configuración del contexto de Spring en las pruebas unitarias y la inyección de dependencias.



Test

2. Pruebas de Integración:

Las pruebas de integración se utilizan para verificar la interacción entre componentes dentro de tu aplicación. En el contexto de Spring, esto podría implicar la interacción entre componentes gestionados por el contenedor de Spring, como controladores, servicios y repositorios.

Spring Test ofrece otras anotaciones útiles, como `@MockBean` para simular componentes dependientes y `@AutoConfigureMockMvc` para probar controladores web.



Test

3- Pruebas de extremo a extremo:

También puedes realizar pruebas de extremo a extremo para verificar el comportamiento de la aplicación en un entorno similar al de producción. En Spring, puedes utilizar herramientas como [RestTemplate](#) o [WebTestClient](#) para realizar solicitudes [HTTP](#) y validar respuestas en pruebas de extremo a extremo.





Empaquetamiento

El empaquetamiento se refiere a **la forma en que se construye y distribuye una aplicación Spring en un archivo ejecutable**, como un archivo **JAR** o un archivo **WAR**. Spring ofrece diferentes opciones de empaquetamiento para adaptarse a los diferentes requisitos de implementación.





Empaquetamiento



1- Archivo JAR ejecutable :

Puedes empaquetar una aplicación Spring en un archivo **JAR** ejecutable que incluya todas las dependencias necesarias y que pueda ser ejecutado directamente mediante el comando `java -jar`. Para ello, se utiliza el plugin **Maven** correspondiente para generar el **JAR** ejecutable.





Empaquetamiento



2- Archivo WAR :

Si estás desarrollando una aplicación web basada en Spring, puedes empaquetarla en un archivo **WAR** (Web Application Archive). Este archivo contiene la aplicación web, junto con las dependencias y los recursos necesarios para ser desplegados en un servidor de aplicaciones compatible con **Java EE**.





Empaquetamiento



2- Empaquetamiento personalizado :

En algunos casos, es posible que necesites un empaquetamiento más personalizado para cumplir con requisitos específicos. Spring **ofrece flexibilidad** para personalizar el proceso de empaquetamiento según tus necesidades, utilizando herramientas como el plugin **Maven Shade** o el plugin **Gradle Shadow**.

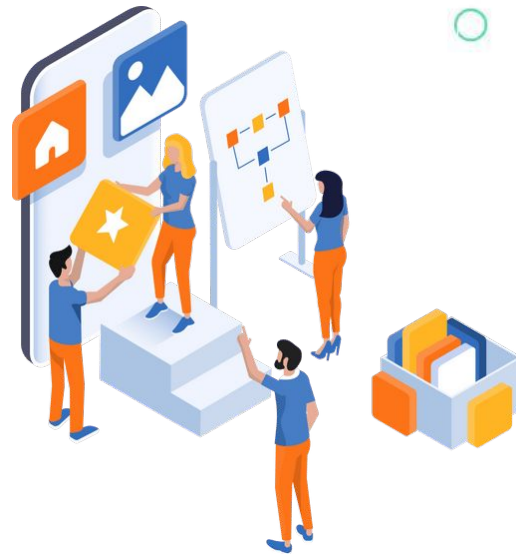


➤ Creando unidades de prueba con Spring



Creando unidades de prueba

Crear unidades de prueba efectivas en Spring es esencial para garantizar que tu aplicación funcione correctamente y se comporte según lo previsto. Spring ofrece una serie de herramientas y enfoques que facilitan la creación de pruebas unitarias para tus componentes.





Creando unidades de prueba



1. Herramientas de Pruebas de Spring:

- **Spring TestContext Framework** : Esta parte de Spring Testing proporciona soporte para cargar y gestionar el contexto de Spring en las pruebas. Te permite ejecutar pruebas en un contexto de Spring configurado, lo que es esencial para la inyección de dependencias y la ejecución de tus componentes en un entorno controlado.
- **SpringJUnit4ClassRunner** : Este corredor de **JUnit** se utiliza para ejecutar pruebas con soporte de Spring. Facilita la carga del contexto de Spring y la inyección de dependencias en las pruebas.
- **@RunWith(SpringJUnit4ClassRunner)**: Anotar tus clases de prueba con esta anotación permite que **SpringJUnit4ClassRunner** ejecute las pruebas, lo que facilita la integración de Spring en tus pruebas unitarias.





Creando unidades de prueba

2. Configuración del Contexto de Prueba:



Un aspecto fundamental de la creación de unidades de prueba en Spring es la configuración del contexto de prueba. Puedes definir un contexto específico para tus pruebas, que puede incluir configuraciones y beans personalizados para simular el entorno de producción. Esto se logra con la anotación `@ContextConfiguration`.



```
@RunWith(SpringJUnit4ClassRunner.class)
@ContextConfiguration(classes = {AppConfig.class})
public class MiClaseDePrueba {
    // ...
}
```



En este ejemplo, AppConfig es la configuración de Spring que se utilizará para cargar el contexto de la prueba. Esto te permite definir un contexto específico para tus pruebas, configurando solo los componentes necesarios para cada caso de prueba.



Creando unidades de prueba



3. Inyección de Dependencias:

Spring se encarga de inyectar las dependencias necesarias en tus componentes de prueba, lo que te permite probar unidades individuales de código de manera aislada.

```
@RunWith(SpringJUnit4ClassRunner.class)
@ContextConfiguration(classes = {AppConfig.class})
public class MiClaseDePrueba {
    @Autowired
    private Servicio servicio; // Spring inyectará automáticamente
    // ...
}
```



La anotación **@Autowired** se utiliza para inyectar automáticamente la dependencia Servicio en la clase de prueba. Esto permite que tus pruebas interactúen con componentes reales de tu aplicación.





Creando unidades de prueba

4. Anotaciones de Prueba: Es común utilizar anotaciones específicas para las pruebas. Algunas de las más utilizadas incluyen:

- **@Test:** Anota un método como una prueba. JUnit utiliza esta anotación para identificar los métodos que deben ejecutarse como pruebas.
- **@Before:** Anota un método que se ejecutará antes de cada prueba. Esto es útil para inicializar el estado necesario para las pruebas.
- **@After:** Anota un método que se ejecutará después de cada prueba. Puedes utilizarlo para liberar recursos o limpiar el estado después de las pruebas.
- **+@RunWith:** Como se mencionó anteriormente, esta anotación se utiliza para especificar el corredor de pruebas que debe utilizarse para ejecutar las pruebas.





Creando unidades de prueba

5. Escribir Pruebas Efectivas:



- Cada prueba debe centrarse en un aspecto específico del comportamiento de tu componente.
- Asegúrate de que tus pruebas cubran todos los caminos posibles a través de tu código, incluyendo casos de éxito y de error. 
- Proporciona datos de prueba que sean relevantes para la funcionalidad que estás probando. Esto facilitará la identificación de problemas cuando las pruebas fallen.
- Las pruebas deben ejecutarse rápidamente y no deben depender de otros componentes o recursos externos.
-  A medida que tu código evoluciona, asegúrate de mantener y actualizar tus pruebas para que sigan siendo efectivas.

LIVE CODING

Wallet Service:

- Vamos a crear una clase `Test` en el paquete `src/test/java`, donde probaremos las funcionalidades de la clase `'UsuarioServicio'` (método `'crearUsuario'`).

Tiempo: 20 minutos

› Empaquetando una aplicacion Spring MVC en un archivo WAR

Empaquetando una aplicación Spring MVC en un archivo WAR

Empaquetar una aplicación Spring MVC en un archivo **WAR** (Web Application Archive) es un paso fundamental en el desarrollo de aplicaciones web en Java. El formato WAR se utiliza para distribuir y desplegar aplicaciones web en servidores de aplicaciones compatibles, como **Apache Tomcat**, Jetty, o servidores de aplicaciones empresariales.





Empaquetando una aplicación Spring MVC en un archivo WAR



Paso 1: Configuración del Proyecto Spring MVC :



Antes de empaquetar tu aplicación en un archivo WAR, asegúrate de que ya tienes una aplicación Spring MVC en funcionamiento. Esto implica la configuración de controladores, vistas, modelos y otros componentes necesarios. Si aún no has desarrollado tu aplicación Spring MVC, este es el primer paso.





Empaquetando una aplicación Spring MVC en un archivo WAR



Paso 2: Dependencias de Maven:

Para empaquetar tu aplicación como un archivo WAR, necesitas agregar las dependencias apropiadas en tu archivo pom.xml. Asegúrate de que tengas las siguientes dependencias de Spring Boot:



```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-tomcat</artifactId>
    <scope>provided</scope>
  </dependency>
</dependencies>
```

Empaquetando una aplicación Spring MVC en un archivo WAR

Paso 2: Dependencias de Maven:

La dependencia **spring-boot-starter-web** proporciona las funcionalidades necesarias para una aplicación web Spring, mientras que la dependencia **spring-boot-starter-tomcat** es necesaria, pero con el alcance `provided`, para indicar que Tomcat (u otro servidor) proporcionará la implementación de servidor incorporada.

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-tomcat</artifactId>
    <scope>provided</scope>
  </dependency>
</dependencies>
```

Empaquetando una aplicación Spring MVC en un archivo WAR

Paso 3: Anotación @SpringBootApplication:

Asegúrate de que tu clase principal de la aplicación Spring MVC esté anotada con **@SpringBootApplication**. Esta anotación combina varias anotaciones importantes, como **@Configuration**, **@EnableAutoConfiguration**, y **@ComponentScan**, que son fundamentales para la configuración automática de Spring Boot.

```
@SpringBootApplication
public class MiAplicacionSpringMvcApplication {

    public static void main(String[] args) {
        SpringApplication.run(MiAplicacionSpringMvcApplication.class, args);
    }
}
```



Empaquetando una aplicación Spring MVC en un archivo WAR



Paso 4: Configuración del Despliegue:

Spring Boot facilita la configuración del proceso de empaquetamiento en un archivo WAR. Debes asegurarte de que tu archivo **pom.xml** esté configurado de la siguiente manera:

La configuración del plugin **spring-boot-maven-plugin** es esencial para que Spring Boot reempaquete tu aplicación como un archivo WAR cuando ejecutes el comando de empaquetamiento.

```
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
      <version>2.7.0</version> <!-- Reemplaza con la versión actual
    </plugin>
  </plugins>
</build>
```

Empaquetando una aplicación Spring MVC en un archivo WAR

Paso 5: Cambio del Tipo de Empaquetamiento:

Para empaquetar tu aplicación como un archivo WAR, debes especificar el tipo de empaquetamiento en tu archivo **pom.xml**. Asegúrate de que el tipo de empaquetamiento sea "**war**". Esto le indica a Maven que debe generar un archivo WAR en lugar del JAR predeterminado.

```
<packaging>war</packaging>
```



Empaquetando una aplicación Spring MVC en un archivo WAR



Paso 6: Empaquetamiento:

Una vez que hayas realizado todos los pasos anteriores, estás listo para empaquetar tu aplicación Spring MVC en un archivo WAR. Utiliza el siguiente comando Maven para empaquetar tu aplicación. Este comando generará un archivo WAR en la carpeta target de tu proyecto.



```
mvn clean package
```

LIVE CODING

Wallet WAR:

- Es hora de empaquetar nuestra aplicación en un archivo WAR.

Tiempo: 15 minutos

¿Alguna consulta?



Resumen

¿Qué logramos en esta clase?

- ✓ Conocer la capa de vista y controladores
- ✓ Aprender a configurar un Controller
- ✓ Controladores Multiacción
- ✓ Aprender a recibir y enviar datos entre un controlador y una vista
- ✓ Desplegar una vista con datos entregados desde el controlador
- ✓ Aprender la implementación de la capa de Servicios
- ✓ Comprender la inyección de servicios en los controladores
- ✓ Aprender la implementación de pruebas y empaquetamiento
- ✓ Comprender el empaquetado de una aplicación Spring en un archivo WAR



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

